

TABLE 1 – TECHNOLOGY REQUIREMENTS

In this document, the project team explains the main technologies selected for the ClassMate app and the reasons behind these choices.

Technology Category	Selected Technology	Why We Need It	Pros (Project-Specific)	Cons / Risks (Project-Specific)
Relational Database	PostgreSQL	We need PostgreSQL to store structured data where relations and constraints matter. This fits our GPA/grade calculation logic and reporting features.	Strong relational model for courses–tasks–grades relationships and joins. Strong relational model for courses–tasks–grades relationships and joins. Supported by Java/Spring Boot.	We must implement more application-level validation, otherwise reminder/notes data can become inconsistent. Schema changes require migrations, that can be error prone if we iterate UI quickly. Using a hosted Postgres creates vendor-lock / cost risk if usage grows beyond free tiers.

Nonrelational Database	MongoDB	Flexible JSON documents, easy to store notes, tags, and reminder metadata without heavy schema design. Good fit for future features like personalized study logs. Works well with microservices: each service can own its own collections and evolve its schema independently.	Allows user inputs for items like notes and reminders to have flexible structure and are more user-friendly. Users will benefit from better UX. Supported by Java/Spring Boot.	Having both Postgres and MongoDB increases operational complexity. We must implement more application-level validation, otherwise data can become inconsistent.
Backend Languages	Java (Spring Boot microservices)	Java will be the language for the logic tier. It will allow the front-end frameworks to access the data layer and enforce business logic.	Spring Boot and other frameworks will allow easy implementation through annotations and conventions to aid development.	Will rely heavily on open-source libraries and frameworks that are subject to change. Changes/updates to one dependency can cascade and complicate upgrades, debugging, and feature expansion.
Frontend Languages	HTML5, CSS3, JavaScript	These languages provide lightweight styling for the app interface.	Efficient for creating ClassMate screens before React integration.	Javascript has no built in component system, so we may write repeated code before migration to React.
Frameworks	Spring Boot	We use Spring Boot to structure each ClassMate backend service with clear domain boundaries and built-in support for REST APIs, validation, data access, and scheduling.	Provides easy setup for microservices, matching our service boundaries. Built-in Spring Data JPA and Spring Data MongoDB	Misconfiguration of multiple services can lead to port conflicts. Auto-configuration may hide complexity, making

			simplify both database connections we use. Spring Scheduler supports reminder notifications and background academic calculations. Strong ecosystem reduces implementation risk and keeps backend consistent for the whole group	debugging harder for new developers. Requires careful version control and documentation so each service stays independent and doesn't become tightly coupled.
Frontend Services	React + React Router + Context API	Interactive UI, handle navigation between the screens, and manage shared state values across entire application with stable API endpoints. Code duplication will reduce.	Allows us to create consistent user experience where navigation and shared data stay synchronized across all screens. React, a component system, will replace repeated UI blocks.	If state grows too large, it may trigger unnecessary re-renders and make the app slower.
Backend Services	RESTful APIs	Microservices in ClassMate depend on RESTful APIs to communicate with each other for sharing data.	Clear separation of services Easy integration with front end and external services.	If API endpoints change, multiple services may break. API gateway may be required which will add complexity to application design.
Hosting Environment	Container orchestration (e.g., Docker Compose) on	A web-based deployment requires hosting of containers so they can be accessed by ClassMate users.	Hosting with a 3rd party avoids the risk of purchasing and maintaining server hardware. Security	As the user base grows, additional costs can be incurred for more compute and storage. The initial

	a 3rd party hosting service.		will also be handled by the 3rd negating the need for an in-house IT department.	Docker Compose setup may need to be upgraded to orchestration systems such as Kubernetes for more efficient scaling. This increases the complexity of the project and comes with a learning curve.
Design Tools	Figma	Figma allowed us to quickly create an interactive, clickable prototype that demonstrates our app's UI, navigation flow, and user experience before actual implementation.	Figma gives our team a shared space to design and test the screens, so we can validate the services workflows before coding.	ClassMate UI may appear functional while not fully matching microservice logic such as grade calculations, reminder scheduling, or task weight rules.
Team Workflow Tools	Trello GitHub	ClassMate depends on for collaboration, version control, planning, and delivery.	Trello gives a simple visual view of tasks, so everyone sees what is assigned GitHub provides version control, branch protection, and pull-request reviews, helping prevent conflicting changes in core modules.	Merge conflicts or incorrect use of branches in GitHub can delay work and introduce bugs into shared microservice code.

TABLE 2 – LEARNING PLAN

Team Member	Project Responsibility	Current Skill Level (%)	Skills Needed to Learn	Learning Plan
Nezihe Tekin	Manage front-end state smooth transitions between screens.	75%	Learn best practices for consistent microservice request/response handling.	Practice combining React Router with Context API
Penny	Develop responsive UI screens, build reusable React components, and integrate microservice API calls into the front-end application.	70%	Advanced Figma-to-React workflow, reusable component patterns, and best practices for integrating REST microservice endpoints.	Practise converting finalized wireframes into functional React components and practice building clean API integration layers using axios with proper error, loading, and success states.
James	Lead the development of back-end services and coordination with front end team	70%	Familiarity with Supabase and MongoDB Atlas BaaS, Java/Spring Boot, and API best practices.	Research API best practices, review Supabase.MongoDB Atlas documentation, and review Spring Boot documentation.
Omar	Back-end architecture design and API gateway	75%	Research optimal architecture design and more practice with Java/Spring Boot.	Research API gateways and review Spring Boot documentation.